

Backend Development — Day-by-Day Execution Plan

Persona: Backend Developer, Farm2Market One-Week Sprint Mandate: Stand up the minimum services the three Must-Haves need (pre-harvest buyer matching, warehouse booking, consumer pre-order), and connect every other track's output into a working system.

Stack decision (made before D1 starts, so I'm not deciding this live under time pressure): Node.js + Express for the API layer, PostgreSQL for relational data (farmers, buyers, warehouses, bookings, pre-orders — this data is inherently relational, not document-shaped), a thin service layer so AI/ML and Data Science logic can be swapped in behind a stable interface, hosted on a free-tier instance (Render/Railway) for demo purposes only.

Day 1 — Data Model, Grounded in the PRD

Goal: A data model that reflects the PRD's actual personas and user stories, not a guess.

1. Re-read the PRD's user stories and personas for the three Must-Haves before writing any schema — the model has to serve the farmer, trader/buyer, warehouse operator, and consumer personas as described, not a generic marketplace shape.
2. Sketch the core entities: **farmers**, **buyers** (traders/corporate), **warehouses**, **listings** (a farmer's pre-harvest supply signal), **matches** (listing↔buyer), **bookings** (warehouse reservations), **preorders** (consumer reservations against a listing).
3. Define relationships and status states up front, since PM flagged the pay-after-delivery/trust flow as the most sensitive surface:
 - **bookings.status**: requested → confirmed → stored → released
 - **preorders.status**: reserved → confirmed → fulfilled → cancelled
4. Write the schema in PostgreSQL (migrations, not just a diagram) so it's runnable, not theoretical.
5. Sit with PM to confirm the model against the personas — catch a structural mismatch today, while it's still cheap to change, per PM's own D1 priority.
6. Set up the repo: Express skeleton, environment config, connection to a dev Postgres instance, basic health-check endpoint so there's something running by end of day.

Deliverable by end of D1: Migrated schema covering all three Must-Haves; running skeleton API; sign-off from PM that the model matches the PRD's personas.

Day 2 — Core APIs + Output Contracts

Goal: Real endpoints for the Must-Haves, and a locked contract with Data Science before their output lands on my desk unannounced.

1. Sync with Data Science first thing: agree the exact shape of the harvest-timing estimate output (a date range with a confidence value, decided as the contract) before building anything that consumes it — this is the dependency PM flagged as the day's biggest risk.
2. Build core CRUD endpoints: `POST /farmers`, `POST /listings`, `POST /warehouses`, `GET /warehouses/:id/availability`.
3. Build the pre-order endpoint shell: `POST /preorders` — accepting a listing ID and consumer ID, status defaulting to `reserved`. Leave the harvest-window field as a stubbed placeholder for now since Data Science's real output isn't ready yet — but structure the field so swapping the stub for the real value later is a one-line change, not a rewrite.
4. Sync briefly with UI/UX on what fields the pre-order and matching screens will actually need to display, so I'm not exposing data nobody asked for or missing something they need.
5. Write basic request validation on every endpoint now, while it's cheap — this also gets ahead of Cybersecurity's D3 review instead of leaving it all for them to find.

← Backend_Developer_Sprint_Pla...

s/listings/warehouses; pre-order endpoint
ct with Data Science.

Day 3 — Integration Begins

Goal: Something callable exists connecting AI/ML and Data Science logic to real endpoints — rough is fine, disconnected is not.

1. Pull in AI/ML's buyer-matching function; wire it into `POST /matches` so a listing can actually return ranked buyer matches, not a hardcoded example.
2. Pull in Data Science's harvest-timing output; replace the D2 stub in `preorders` with the real field, matching the contract agreed yesterday.
3. Test both integrations against 2–3 real records from the case-study dataset — not synthetic "it compiles" data, since PM's framing today is "is this integrating against something real or a placeholder."
4. Where the actual output doesn't match what I expected from the D2 contract (a common outcome — contracts drift once code exists), fix it today directly with AI/ML and Data Science rather than working around it silently.
5. Share the first working endpoints with Frontend so their build can start against something real, even if incomplete, ahead of tomorrow's scheduled Frontend-Backend integration.
6. Make the pre-order and booking data models available to Cybersecurity for their review, since that starts today per PM's plan — walk them through the schema directly rather than leaving them to reverse-engineer it.

Deliverable by end of D3: Buyer-matching and pre-order endpoints running against real logic, not stubs; first hand-off to Frontend; data models handed to Cybersecurity for review.

Day 4 — Frontend Integration + Security Follow-Up

Goal: Frontend can actually complete each Must-Have journey against my APIs, end to end.

1. Pair with Frontend through each of the three journeys live: buyer matching, warehouse booking, pre-order. Fix broken contracts or missing fields on the spot rather than filing them for later — today's the scheduled day for this per the plan.

2. Implement the booking status transitions fully (**requested** → **confirmed** → **stored** → **released**) so Frontend can show real state changes, not just a static "booked" label.
3. Check in with Cybersecurity on their deepening review; anything they flag as must-fix on the pre-order/booking flows goes straight into today's work, ahead of anything else, per PM's triage priority.
4. Fix any must-fix security items same-day — input validation gaps, missing auth checks on sensitive endpoints (payment/reservation-adjacent data).
5. Confirm with PM where each journey stands (working / partial / blocked) honestly — no rounding up "almost working" to "working" in the standup.

Deliverable by end of D4: All three journeys functional against real APIs from Frontend's side; Cybersecurity's critical findings fixed same-day, not deferred.

Day 5 — Integration Testing + UX QA Support

Goal: Verify, don't build — today is about proving what exists actually works, not adding to it.

1. Run each of the three Must-Have journeys myself, end to end, before anyone else tests them — catching my own bugs first is faster than having them found downstream.
2. Support whoever PM assigns to test flows I built (deliberately not me, per PM's "fresh eyes" approach) — be available to explain behavior, not defensive about bugs found.
3. Specifically re-verify the pre-order flow shows a real Data Science-derived harvest window, not a leftover stub — PM flagged this exact risk on D5, and it's my responsibility to confirm it's actually resolved, not assumed.
4. Log every bug found today as a ClickUp card immediately with severity, per PM's process — don't fix from memory later.
5. Triage: fix anything genuinely quick today; anything larger goes onto the **Fix Before Demo** list PM is consolidating for D6.
6. Give PM an honest technical read on pre-order specifically — if the booking-to-fulfillment status flow isn't holding up reliably under testing, say so plainly, since PM needs that to make the D5 stretch-goal call.

Deliverable by end of D5: All three journeys tested end-to-end by me and at least one other person; bugs logged and triaged; an honest status report on pre-order's reliability for PM's go/no-go call.

Day 6 — Fix List Execution + Go/No-Go Support

Goal: Close everything on the fix list — no new work, no new scope.

1. Work the **Fix Before Demo** list top to bottom by severity, updating ClickUp status as each item closes.
2. Confirm with Cybersecurity that nothing security-flagged remains open by midday, matching PM's checkpoint.
3. Support PM's go/no-go checklist directly: demonstrate each Must-Have journey live when asked, and give a clear yes/no/with-caveats on the backend's readiness for each.
4. If pre-order is marked "with caveats" for the demo, agree the specific fallback with PM now (e.g., a pre-seeded example instead of live end-to-end) so there's no improvising tomorrow.
5. Join the rehearsal PM runs; fix anything the rehearsal exposes, then stop — scope is frozen after today.

6. Do not start anything new, even something quick and tempting — it goes to **Post-Launch** per PM's freeze.

Deliverable by end of D6: Fix list closed; backend readiness confirmed per journey for the go/no-go checklist; rehearsed and stable for D7; scope frozen.

Day 7 — Demo Support + Retro

Goal: Support a smooth demo, then contribute honestly to the retro and backlog.

1. Be on standby during the demo to explain backend behavior if asked, and to catch anything unexpected in real time — but the goal today is stability, not last-minute fixes.
2. Confirm before the demo starts that the deployed instance is up and seeded with clean demo data (not leftover test junk from D5–D6 testing).
3. During the retro, give a specific, honest account of what was hard this week from a backend standpoint — e.g., contract drift with AI/ML and Data Science outputs was the recurring friction point, not the coding itself — rather than a generic "it went well."
4. Flag anything backend-related that got descoped or shipped with caveats (e.g., if pre-order's status flow needs hardening) directly into PM's **Next Sprint Backlog**, with enough technical detail that whoever picks it up next sprint doesn't have to reverse-engineer the gap.

Deliverable by end of D7: Stable demo; honest retro input specific to backend friction points; clear, technically detailed backlog items handed off for next sprint.